

CoddSpeed: Hardware Accelerated Query Processing in Microsoft Fabric

Matteo Interlandi
Nicolas Bruno
Brandon Haynes
Carlo Curino
Rathijit Sen
Yinan Li
Kaushik Rajan
Bailu Ding
Lukas M. Maas
Wei Cui
Kevin Gaffney
Mingsheng Hong
Brian Kroth
Sampath Rajendra
Peng Cheng
Surajit Chaudhuri
Microsoft
Redmond, WA, USA

Johannes Gehrke
Raghu Ramakrishnan
Lidong Zhou
Momin Al-Ghosien
Craig Peeper
Marius Dumitru
Conor Cunningham
Kevin Bocksrocker
Prasanna Sundar
Ron Boskovic
YongChul Kwon
Marko Zivanovic
Runbin Shi
Krystian Sakowski
Denis Lamtsov
Josep Aguilar Saborit
Microsoft
Redmond, WA, USA

Krish Srinivasan
Adarsh Kapil
Andrew Putnam
Anudeep Kambapu
Aparna Konduri
Aaron Landy
Aaron Moore
Aaron Pitman
Artem Oks
Ashit Gosalia
Babu Kandimalla
Blake Pelton
Bogdan Crivat
Cesar Galindo-Legaria
Chidamber Kulkarni
Jesus Camacho
Rodriguez
Microsoft
Redmond, WA, USA

Evgeny Babin
Hans Lehnert Marino
Igor Konev
Israel Arroyo
Luis Penaranda
Mandar Datar
Morten Borup Petersen
Nicholas Simmons
Parminder Poonian
Pravija Danda
Rob Rydberg
Esteban Calvo
Ronak Bajaj
Sai Sumanth Kammal
Shetty
Sharanya Bhat
Zach Iracheta
Microsoft
Redmond, WA, USA

Abstract

Over the past three decades, Microsoft has developed several multi-billion-dollar data management products. We have now unified them into Microsoft Fabric, a comprehensive SaaS suite for data management based on a modern cloud-native design that delivers best-in-class analytics performance on CPUs. Meanwhile, Microsoft's AI investments have transformed our data centers. Hardware accelerators are now plentiful and surpass traditional CPU servers by orders of magnitude in compute, memory, and networking capabilities. *Running analytics on hardware accelerators is therefore a business imperative.* In this paper, we describe a multi-year effort to take Fabric to the next level by enabling analytics execution on state-of-the-art hardware accelerators. Given the rapid evolution of hardware, we have made *hardware flexibility* our guiding principle. We validate our architectural flexibility by demonstrating how we can run Fabric engines (e.g., Data Warehouse) on a variety of compute and network accelerators (e.g., GPUs, FPGAs, ASICs, NVLink, InfiniBand). We present our most mature implementation, a GPU-based execution engine derived from our Tensor Query Processor, and report key results from a novel data movement system leveraging NVLink and InfiniBand. This version of Fabric outperforms its CPU counterpart by over an order of magnitude across

a host of production and benchmark scenarios (e.g., delivering up to 30× speedup on TPC-H 1TB). We share the lessons learned in building one of the first systems in the era of *accelerated analytics*.

CCS Concepts

• **Information systems** → **Database query processing; DBMS engine architectures.**

Keywords

GPU databases, Coprocessors; Fast shuffle; Substrait; FPGA; ASIC

ACM Reference Format:

Matteo Interlandi, Nicolas Bruno, Brandon Haynes, Carlo Curino, Rathijit Sen, Yinan Li, Kaushik Rajan, Bailu Ding, Lukas M. Maas, Wei Cui, Kevin Gaffney, Mingsheng Hong, Brian Kroth, Sampath Rajendra, Peng Cheng, Surajit Chaudhuri, Johannes Gehrke, Raghu Ramakrishnan, Lidong Zhou, Momin Al-Ghosien, Craig Peeper, Marius Dumitru, Conor Cunningham, Kevin Bocksrocker, Prasanna Sundar, Ron Boskovic, YongChul Kwon, Marko Zivanovic, Runbin Shi, Krystian Sakowski, Denis Lamtsov, Josep Aguilar Saborit, Krish Srinivasan, Adarsh Kapil, Andrew Putnam, Anudeep Kambapu, Aparna Konduri, Aaron Landy, Aaron Moore, Aaron Pitman, Artem Oks, Ashit Gosalia, Babu Kandimalla, Blake Pelton, Bogdan Crivat, Cesar Galindo-Legaria, Chidamber Kulkarni, Jesus Camacho Rodriguez, and Evgeny Babin, Hans Lehnert Marino, Igor Konev, Israel Arroyo, Luis Penaranda, Mandar Datar, Morten Borup Petersen, Nicholas Simmons, Parminder Poonian, Pravija Danda, Rob Rydberg, Esteban Calvo, Ronak Bajaj, Sai Sumanth Kammal Shetty, Sharanya Bhat, Zach Iracheta . 2026. CoddSpeed: Hardware Accelerated Query Processing in Microsoft Fabric. In *Companion of the International Conference on Management of Data (SIGMOD Companion '26)*, May 31-June 05, 2026, Bengaluru, India. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3788853.3803077>



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGMOD Companion '26, Bengaluru, India*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2450-3/2026/05
<https://doi.org/10.1145/3788853.3803077>

1 Introduction

Data analytics is central to most business operations and strategic decision making and high performance querying is key for our customers across Lakes, Warehouses, and BI reporting. Moreover, the shift towards agent-mediated access to data will produce bursty and high-volume querying with an increased premium on latency [46].

Over the last 30 years, we have built and operated several analytics services at Microsoft (e.g., SQL Server, Synapse Data Warehouse, Synapse Spark, and PowerBI). Microsoft Fabric [48] brings these capabilities together in a single unified SaaS suite, simplifying data-access, security, and billing. The analytics services in Fabric expose different user experiences (e.g., Notebooks, ETL, complex analytical SQL, BI dashboards) yet share two key characteristics: (1) they support high-performance exploration of relational data, and (2) they are built to run on x86 CPUs. While the focus on relational querying is likely to remain relevant for years, the slowing of Moore's law and saturation of software performance gains on CPUs make an exclusive bet on CPUs for analytics increasingly risky. This is particularly evident given the rapid proliferation of *hardware accelerators* in modern data centers, an industry wide phenomenon fueled by an AI infrastructure spending estimated to exceed 5 trillion dollars by 2030 [56]. While we do not know which specific mix of compute (GPUs, FPGAs, ASICs [73], NN coprocessors [19]) and scale-out interconnects (Ethernet, NVLink [59], Infinity Fabric [5], Infiniband [58], CXL [22], or on-chip optics [36]) will deliver the best (price-)performance for analytics, we must ready our database software stack to take full advantage of them.

This presents us with a challenge: how do we offer customers a familiar and broad product surface while adapting to a rapidly-evolving hardware environment that we can no longer ignore? We address this challenge by introducing two abstractions: a **Coprocessor Abstraction Layer (CAL)** that encapsulates relational query processing using standardized plans, and a **Data Abstraction Layer (DAL)** that isolates users and services from the specific mechanics used to cache and shuffle inputs and intermediate data [50]. We implemented CAL engines for GPUs, FPGAs, and ASICs. Similarly, our DAL implementation allows indirection over NVLink, Infinity Fabric, Infiniband, PCIe and Ethernet. For conciseness, we focus our discussion on Fabric Data Warehouse (or Fabric DW) as the host engine, our GPU-based CAL implementation, and a diverse set of DAL configurations (e.g., CPU/GPU cache, NVLink, Infiniband for networking). In Section 3.5, we summarize the results we obtained using alternative CAL implementations.

Our approach to accelerating relational queries, called *CoddSpeed*, introduces an execution model where *query fragments* are offloaded to one or more coprocessors in a cost-based manner, while maintaining compatibility with existing architectures. We discuss the challenges of memory constraints, query optimization strategies, and the runtime execution model that coordinates between host systems and coprocessors. The GPU engine component is a hardened and optimized version of the Tensor Query Processor (TQP) [9, 32]. Our multi-year endeavor has yielded 8–12× performance gains on TPC-H 100GB and 15–20× speedup on customer workloads. While details of our DAL component are beyond the scope of this paper, we present scale-out experiments over 1TB TPC-H using both CAL and DAL in concert. Interestingly, the relative abundance

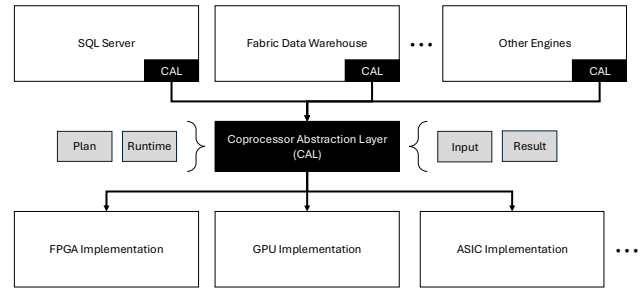


Figure 1: The Coprocessor Abstraction Layer (CAL) mediates integration of data engines with disparate hardware devices.

of network bandwidth allows us to scale nearly linearly, so our performance relative to scale-out baselines grows further to 25–30× for TPC-H 1TB running across multi-GPU/node settings. Finally, we describe how this architecture now serves as a foundation for advanced research efforts [35, 44, 79]. While not all are fully productized, they chart a path towards significant gains in performance, cost efficiency, and generality. This continuous cycle of innovation, reinforced by promising hardware trends, strengthens our confidence that we are on the right path to deliver substantial benefits to our customers through *accelerated analytics*, riding the current AI-inspired wave of investment in accelerators, and beyond.

2 Design Principles for Hardware Acceleration

Building industrial-strength coprocessors and data movement engines is a challenging task. To limit scope and bound implementation effort, we adopted the following design principles:

Simplicity. CAL must be consumable by different single-node and distributed host data engines (e.g., SQL Server or Fabric DW) and flexible enough to target different hardware environments (e.g., GPUs, FPGAs). This is achieved through a simple interface to be implemented and consumed by many participants (Section 2.1).

Host-based optimization. We expect the host engine to apply common optimizations (e.g., join ordering, filter push-down) before offloading work to coprocessors. Optimizations applied by a coprocessor are optional and typically data-dependent (Section 2.2).

Coprocessors offer differing functionality. Coprocessors are not expected to implement all the surface area offered by a host engine, such as, for example, outer join support, or specific user-defined functions (Sections 2.3 and 2.4).

Coprocessors may fail. When failures occur, the host must be able to recover. For example, the working set of a query fragment sent to a coprocessor is assumed to fit into its local memory. Otherwise, the coprocessor may fail and revert execution to the host (Section 2.5).

Amdahl's Law. As the performance of queries improves thanks to accelerators, data ingestion and movement across nodes become the new bottleneck. To accelerate the I/O path we designed a separate, complementary service alongside coprocessors (Section 2.6).

2.1 Hardware-Agnostic Coprocessor API

The Coprocessor Abstraction Layer (CAL) is based on a hardware-agnostic API that works seamlessly across various hosts. CAL hides the specifics of the underlying hardware, enabling deployments across platforms that include single-node and distributed variants.

Figure 1 provides a high-level design for CAL, which consists of four components: Plan, Runtime, InputSource, and Resultset (a full description of the API is beyond the scope of this paper).

When a host engine pushes a query fragment to a coprocessor, it uses a Plan object. Each Plan: (1) encapsulates query semantics via a graph of relational and scalar operators, (2) uniquely identifies the data constructs to be operated over (e.g., tables, columns), and (3) tracks the life-cycle of query fragments. Query fragments are serialized using Substrait [52], an open-source standard for cross-language serialization of relational algebra. Once a Plan is created, hosts use Runtimes to encapsulate its execution units. Hosts may spawn multiple Runtimes for a given Plan to implement parallelism. Data is wrapped in an InputSource instance and passed in its native format (e.g., Parquet or other native columnar formats) to a Runtime. This allows the coprocessor to decide how to consume such data (e.g., by transcoding it into an internal representation or directly consuming it [35, 45]). Correspondingly, columnar Resultset instances describe how the output is returned to the host. A Resultset comprises metadata (e.g., physical types) and on-device pointers to data so that hosts may consume output in a zero-copy manner. The Resultset is intentionally generic, to allow advanced functionalities such as GPUDirect RDMA [57] when supported by the Data Abstraction Layer (DAL).

2.2 Optimizing with a Minimalist Coprocessor

Implementing a coprocessor that efficiently supports multiple host engines and hardware is challenging; implementing one that can optimize query plans is even harder. Because of this practical consideration, we separated *compile-time optimizations* (i.e., typical relational optimizations such as join ordering and filter push-down) and *run-time, data-dependent optimizations*. One key design choice for CAL is that hosts assume that any pushed query fragment is executed *as-is*. Therefore, all critical compile-time optimizations should be applied before the fragment is pushed. For runtime optimizations, a coprocessor implementation can optionally elect to leverage runtime information such as input format, data types, and device characteristics to further optimize the (local) execution of the query. This design allows us to avoid re-implementing a full-fledged query optimizer while still enabling efficient query execution over a diverse set of target hardware accelerators. For example, optimizations in our GPU-based CAL implementation include: (1) input-format-based reordering of operators materializing join outputs, (2) fast join implementation for PK-FK joins, and (3) optimized hash table format for unique inputs. Section 3.4 provides implementation details for the GPU coprocessor.

2.3 Coprocessor Capabilities

A key design requirement is to allow arbitrary host engines (e.g., SQL, Spark, PowerBI) to delegate portions of query computation to coprocessors with diverse abilities. We therefore need a robust mechanism to negotiate which plans can be handled by a coprocessor. CAL exposes a set of *capabilities* that characterize the space of plans that can be offloaded to coprocessors by the query optimizer (Section 3.1). Each coprocessor exposes two types of capabilities:

Local Capabilities. These represent the ability of a coprocessor to evaluate specific scalar and relational operators. For example,

a coprocessor might handle filters and aggregation but not join variants. Among scalar operators, a coprocessor might be able to evaluate comparisons and LIKE operators, but not other intrinsic functions like UPPER or COS. To capture this, for a given coprocessor, a local capability LC is formalized as a function $LC : \text{Node} \rightarrow \text{bool}$, where Node corresponds to each relational or scalar operator node in a plan. If $LC(\text{GroupBy}) = |\text{GroupBy.groupingCols}| < 4$, then the coprocessor is capable of handling GroupBy operators with fewer than 4 grouping columns. If $LC(\text{LeftSemiJoin}) = \text{false}$, then the coprocessor cannot handle left semi-join operators (however, a host might still offload a semi-join if it can transform the operation into a join over a distinct operator and those alternative operators are handled). We say that a plan P satisfies a local capability LC whenever each operator node $N \in P$ does, or if $\forall N \in P : LC(N)$.

Global Capabilities. These describe more complex computational restrictions in the coprocessor. An example of such a capability is a limit on the complexity of the computation that is being offloaded. Hardware limits might rule out expressions beyond a certain size (e.g., at most k conjuncts in a scalar expression) or impose restrictions on the sizes of input and intermediate data. Formally, we define a global capability as a function $GC : \text{Plan} \rightarrow \text{bool}$. Currently, our algorithms on hardware accelerators require that the query input and working set fit in the high-bandwidth device memory (HBM). We then adopt a global capability as follows.

For a plan P over tables T and a coprocessor with memory M , $GC(P) = \sum_{t \in T} \text{size}(t) + \sum_{O \in P} \text{workingMemory}(O) \leq M$. This capability disallows traditional execution alternatives that handle larger-than-memory data by spilling results out of memory. Spilling algorithms on coprocessors would relax this constraint, which we leave for future work. Note that this capability is enforced at compile-time using size estimates and can fail at runtime (see Section 2.5 for a description of how we handle this scenario).

Current status. Capabilities are specified using Substrait and leveraging extensibility points to describe constraints on types, scalar, and relational operators. At this point, capability specifications are not automatically consumable by hosts. Instead, database implementers follow the capabilities specification and modify the optimizer to push query fragments that are allowed. We are currently exploring approaches to perform automatic capability negotiation.

2.4 Partitionable Execution Model

A common challenge when using coprocessors is their limited on-board memory. The reason, encoded in the global memory capability discussed earlier, is that all data for an offloaded query fragment must fit in coprocessor memory. This approach limits the types of query fragments that can be executed on coprocessors to only those operating on data small enough to fit in available memory.

We can relax this restriction using distributed plans. Each task of a distributed plan is an independent unit that operates on smaller, partitioned subsets of data. We can then push distributed query fragments to coprocessors, operating over data partitions, even when the full data does not fit in the coprocessor memory. We next describe a complementary approach, inspired by this distributed model, that can also be applied to individual nodes *within* a distributed plan to further extend coprocessor utilization. Consider the query below, for which table F exceeds the coprocessor memory:

```

SELECT F.x, D.y
FROM F JOIN D
ON F.fk = D.pk

```

Suppose that we partition F into non-overlapping *chunks* F^k . A good property of the above query is that we can obtain its result by (1) evaluating the query multiple times by replacing F with each chunk F^k , and (2) combining individual results. If F does not fit into the coprocessor memory but D and F^k do, we can still evaluate the query by sending D to the coprocessor, progressively sending each F^k to coprocessor memory, and incrementally obtaining the overall result by combining partial results. We formalize this idea below.

Definition 2.1. A query Q on tables $T_1, \dots, T_n$ is *partitionable* over T_i if $Q[T_1, \dots, T_i, \dots, T_n] = \cup_k Q[T_1, \dots, T_i^k, \dots, T_n]$ for $T_i = \cup_k T_i^k$.

Using this definition, the query above is partitionable over F , while the following queries are not, because we do not obtain the original result by concatenating partial results:

```

Q1 = SELECT F.x, D.y      Q2 = SELECT F.x, COUNT(*)
    FROM D LEFT JOIN F
    ON D.dk = F.fk        FROM F
                           GROUP BY F.x

```

However, we can *decompose* the GROUP BY operator in query Q2 above into a local (SQ) and global pair as follows:

```

Q2' = SELECT F.x, SUM(C)
      FROM (SELECT F.x, COUNT(*) AS C
            FROM F "LOCAL" GROUP BY F.x) SQ
      GROUP BY F.x

```

After decomposition, SQ becomes partitionable over F and the original query Q2 can be evaluated as follows. First, SQ is progressively evaluated in the coprocessor by iteratively processing each partition. As each partition completes, the host accumulates intermediate results. Once all partitions of SQ have been processed, the host executes the outer (global aggregate) query fragment. More complex alternatives are possible (e.g., overlapping input and processing, or buffering intermediates from local aggregates and push global aggregates to coprocessors). We defer these options to future work.

This partitionability scheme leverages query fragments that are partitionable over a table T and result in the memory constraint of the coprocessor being satisfied by individual fragments.¹ These fragments are offloaded to the coprocessor one at a time, and results are aggregated. Coprocessors *do not know* how partial results would be put together, which is the responsibility of the host engine. A benefit of this approach is that it covers an interesting subset of queries that appear frequently in analytical scenarios, such as star-joins of a large fact table and small dimension tables with subsequent aggregation. Additionally, it enables the execution of complex queries by offloading *partial fragments* without requiring us to push the whole query or nothing at all. This enables fine-grained fault tolerance, in which a single fragment can be re-executed in the host if it fails in the coprocessor. Finally, the coprocessor engine and algorithms do not need to consider spilling or be aware of scheduling decisions, further simplifying the overall approach.

2.5 Query Fallback

Query fragments sent to the coprocessor might (rarely) fail for a variety of reasons. For example, the coprocessor can run out of

¹Partitionability can be extended to multiple tables when we can partition them in a way that results in collocated n -ary operations, similar to distributed processing ideas.

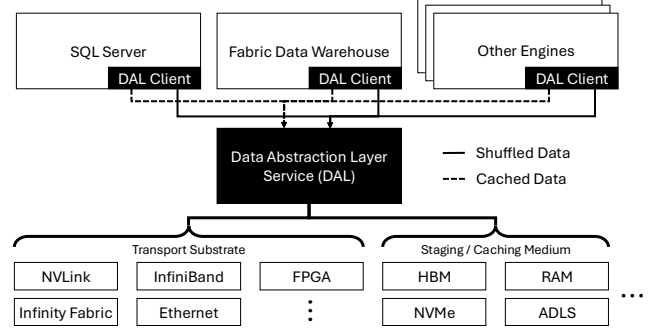


Figure 2: Data engines cache and shuffle data by interacting with the data abstraction layer (DAL).

memory or hit unexpected conditions at runtime. If that happens, we recover by having the host engine reprocess the failed fragment. For this purpose, we rely on the fact that (by design) *each* individual execution in the coprocessor comes back with either a full result (for the partition) or a failure. This simplifies recovery as we can reprocess a failed fragment without worrying about partial results being consumed by the host before the failure. This *fallback* mechanism is compatible with our target frontend engines (e.g., Fabric DW) and enables a finer-grained fault tolerance, in which failures can be handled without having to reissue the full query.

2.6 Data Movement as a Service

As coprocessors improve compute throughput, overall query performance becomes constrained by *data access and movement* (i.e., fetching data from local caches or remote storage, and shuffling intermediate results across nodes). We addressed both aspects using a data abstraction layer (DAL) relying on these principles:

Local writes and remote reads are the *unified paradigm* for enabling both efficient data exchange in distributed environments and support for caching. This paradigm decouples producer and consumer scheduling, allowing producer coprocessors to complete their writes independently without requiring consumer participation, and therefore improving overall utilization. Since objects written to DAL can persist beyond the lifetime of a single query, future queries can benefit from caching and result reuse.

Tiering. In practice, DAL mirrors the CAL approach: we abstract complexity and heterogeneity behind a dedicated service layer and API. DAL, summarized in Figure 2, manages both local (e.g., caches spanning GPU memory, CPU memory, or disks) and remote data (e.g., external storage such as Data Lakes). Beyond access, DAL handles data placement decisions, including promoting frequently accessed data to faster tiers (e.g., GPU memory), evicting less used data to lower-cost storage, and orchestrating data movement across nodes using heterogeneous transport mechanisms such as Ethernet, NVLink, and InfiniBand. A host engine interacts with this service through the DAL API, without needing to be aware of low-level details of caching, transport selection, or topology.

Zero-copy data access is of paramount importance to achieve both optimal on-coprocessor memory utilization as well as performant data movement. Analogously to CAL, DAL produces and consumes on-device pointers whenever possible to maximize opportunities for zero-copy execution and data movement.

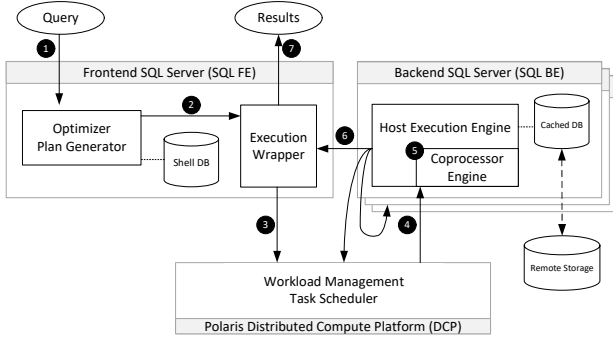


Figure 3: Life of a query in the Fabric Data Warehouse.

3 Query Processing using Coprocessors

CAL and DAL provide general abstractions that enable different combinations of hosts and hardware to work together. In the remainder of the paper, we focus on one such setup using Microsoft Fabric DW [17] as the host engine and describe our extensions to offload computation to hardware coprocessors. Figure 3 illustrates the steps involved in returning results for an input query in Fabric DW. The input query is initially sent to the Frontend SQL Server Engine (SQL FE, step ①) and compiled using a *shell database* as the source of metadata, table statistics, and authentication. The query optimizer is extended to generate distributed plans that can push computation to coprocessors (Section 3.1), and the resulting plan is handled by a thin execution wrapper in the SQL FE (step ②). The plan is then sent to the Polaris Distributed Compute Platform (DCP) [3] (step ③), which performs distributed workload management and scheduling, forwarding plan fragments to Backend SQL Server instances (SQL BE, step ④). Plan fragments received by SQL BE nodes are executed (step ⑤) using extensions to deal with coprocessors (Sections 3.2 and 3.3). Results of backend executions are moved according to Polaris DCP’s data movement directives among SQL BE nodes, and execution metadata is sent back to the DCP (step ⑥). For the last distributed step, results are sent to the SQL FE, which aggregates and sends them back to the client (step ⑦). In the rest of this section we describe in additional detail the extensions to Fabric DW required to evaluate queries using coprocessors. For the sake of brevity, we will be using the GPU implementation of CAL. We refer readers to Sections 3.5 and 4.3 for a discussion and results using other types of coprocessors.

3.1 Query Optimization

Query optimization in Fabric DW is based on the *Cascades* framework [30], which systematically explores the plan search space using a collection of transformation rules, leverages the algebraic representation of query fragments, and results in an integrated and extensible approach to obtain efficient execution plans.

To represent the plan fragments to be sent to coprocessors, we introduce a new physical operator called Coprocessor. The query fragment below a Coprocessor node (extending to leaf nodes or data movement operations) is then pushed down to the coprocessor. We define new logical properties (which capture local and global capabilities of the coprocessor over logical trees), and physical properties (which indicate whether a plan is executed inside a coprocessor and, if so, whether it is partitionable over some table or

intermediate result T). A new *enforcer* rule offloads query fragments to a coprocessor whenever their logical properties allow it. For plans optimized in the context of a coprocessor, a different set of rules is triggered (e.g., merge joins are not generated if the coprocessor cannot support them), and a specialized cost model is used to compare alternative plans. Rather than detailing the full optimization process, we next summarize the extensions to enable coprocessor support and highlight some key features of the optimizer.

Integrated Optimization. Local and global coprocessor capabilities affect which query fragments can be delegated to coprocessors. We thus cannot rely on a phase separate from cost-based optimization to make those decisions. It is crucial that the optimizer explores an integrated search space considering coprocessor placement whenever possible. Consider a query fragment $S \bowtie_p (GroupBy_{\{a\}}(T))$ and suppose that T does not fit in coprocessor memory. The presence of the GroupBy operator makes the query not partitionable over T , so the largest fragment that can be pushed to the coprocessor is a local aggregate over T . When coprocessor placement is integrated into search, we can split the aggregate into local and global components, and pull the global aggregate above the join, obtaining $GroupBy_{\{a\} \cup key(S)}(S \bowtie_p (LocalGroupBy_{\{a\}}(T)))$. This alternative might not be the optimal in a traditional execution environment without coprocessors, but when coprocessors are available, it enables both the join and local aggregate to be pushed down, resulting in a more efficient plan overall.

Principle of Optimality. Analogous to *interesting orders* in traditional optimization, global coprocessor capabilities can violate the principle of optimality (which states that the optimal plan is built from optimal sub-plans). Consider joining tables R (size 10) and S (size 1). A plan $P_S = R \bowtie S$, partitionable over S , has a memory requirement of $10 + \epsilon$ (10 for R and a small ϵ for each partition of S , ignoring for simplicity hash-table memory overhead). Alternatively, a plan $P_R = S \bowtie R$, partitionable over R , might use only $1 + \epsilon$ of memory. If P_S is cheaper than P_R , the optimizer would pick P_S using the principle of optimality, and discard P_R from further consideration. However, if the query later joins a third table T , the plan that joins the three tables together using P_S as subplan might exceed the coprocessor’s memory limit, preventing the entire 3-way join from being pushed down. The issue arises because P_S is faster than P_R but uses more coprocessor memory. Retaining both P_R and P_S would allow the optimizer to find the best alternative overall. In general, we need to consider the *skyline* of plans with respect to cost and memory (generalizing sort orders [69]) to prevent violations to the principle of optimality. In our implementation, we address this challenge through careful property manipulation and by relying on resource estimation to adjust degree of parallelism of plans. We omit further details due to space constraints.

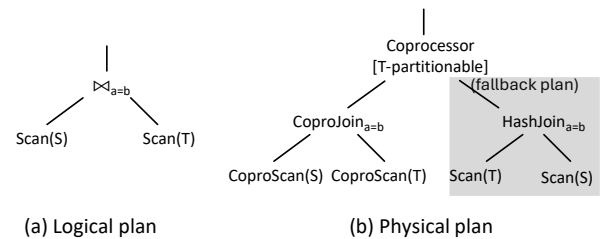


Figure 4: Compilation of a logical plan using coprocessors.

Dual plans below Coprocessor operators. To support fallback execution (where plan fragments are executed on the host when coprocessors fail to produce results for a partition), the optimizer generates *two* plans for each fragment that is pushed to a coprocessor. Figure 4(a) shows a logical plan joining tables S and T . When the optimizer considers pushing the join to a coprocessor, the enforcer rule inserts a physical Coprocessor operator at the top, containing two copies of the original logical plan from Figure 4(a). The first child corresponds to the fragment pushed to the coprocessor, and the second one to the fallback plan. During optimization, each child is processed under a different context, with different implementation rules and cost models. The resulting plan, shown in Figure 4(b), has a coprocessor on a plan partitionable over T (left subtree) and its corresponding fallback plan (right subtree). This example shows that both branches can be optimized differently under each context.

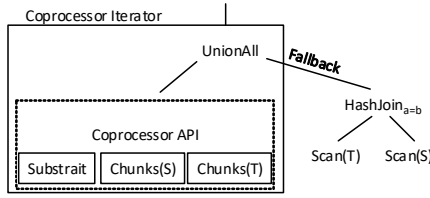


Figure 5: The coprocessor iterator.

3.2 Query Execution in the Host

The final compilation stage transforms physical operators into executable iterators. Creating a coprocessor iterator involves a few steps. First, we take the pushed query fragment (left child in Figure 4(b)) and translate it into Substrait [52]. We then identify the data sources for the plan (tables S and T in Figure 4(b)) and generate iterators that return *chunks* of data (e.g., a few Parquet files). The coprocessor iterator, illustrated in Figure 5, takes the union of the offloaded results and the optional fallback results.

Internally, the iterator operates as a state machine that goes through several stages while interacting with the coprocessor API (see Figure 6). When the coprocessor iterator is opened in the *Initial* state, the Substrait plan is sent to the coprocessor via the *CreateQueryPlan* API call, and the state transitions to *NonPart*. In this state, the host sends all chunks of non-partitionable tables to the coprocessor using the *SendChunk* API call, after which it transitions to state *Setup*. In this state, the *Prepare* API call informs the coprocessor that all non-partitionable data is available, allowing it to set up internal data structures for subsequent execution calls (for example, creating hash tables for joins²). After preparation, the state transitions to *Partitionable*, where the iterator sends a chunk of the partitionable data to the coprocessor. It immediately transitions to the *Results* state, triggering execution on the coprocessor via the *Execute* API call and returning results to the parent iterator. Once all results for the current *Execute* call are sent to the parent iterator, the state reverts to *Partitionable* to process the next chunk of data. During this back and forth between *Partitionable* and *Results* states, the iterator tracks any partitionable chunks that failed to execute. When there are no more chunks to process, the state changes

²*Prepare* calls ensure that common fragments are evaluated once and cached, so subsequent executions reuse shared information like hash tables.

to *Done* if fallback is not needed, or *Fallback* otherwise. During *Fallback*, the iterator forwards *GetNext* calls to the host fallback plan, which processes only the chunks that previously failed and returns the corresponding results. Because of the partitionability property and the fact that coprocessor results are fully returned or the API call fails, the overall results are still correct in the presence of coprocessor failures. Due to space constraints, we omit details on how we handle parallelism via multiple runtimes, or interactions between fallback and ordered results.

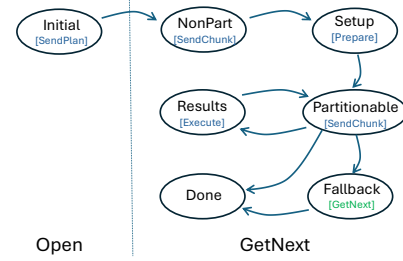


Figure 6: State machine for the coprocessor iterator.

3.3 Query Execution in the Coprocessor

We now explain the life cycle of a fragment pushed to a coprocessor. We will be using our GPU implementation of CAL as a reference.

Plan Generation. Upon query execution, the first API being called by the host engine is *CreateQueryPlan*: this creates a *Plan* object from an input Substrait plan (see Figure 7(a)). Internally, the coprocessor parses the Substrait plan and generates an in-memory representation of the query fragment (or *Physical plan* in Figure 7(b)). At this stage, the coprocessor has only seen the plan but not the data yet. The generation of the executable plan for the GPU is deferred until the data is registered in the coprocessor, so that the optimal execution strategy can be assembled given input characteristics.

Runtimes. The execution unit of a query fragment in a coprocessor is a *Runtime*. Each fragment has at least one *Runtime*, but possibly more based on how many chunks of the partitionable table we want to process concurrently, and how many more non-partitionable tables exist in the plan.

Data Registration and Binding. Once a *Plan* is generated, and the *Runtime(s)* created, the host engine registers the input data into the coprocessor. This step follows the approach described in the

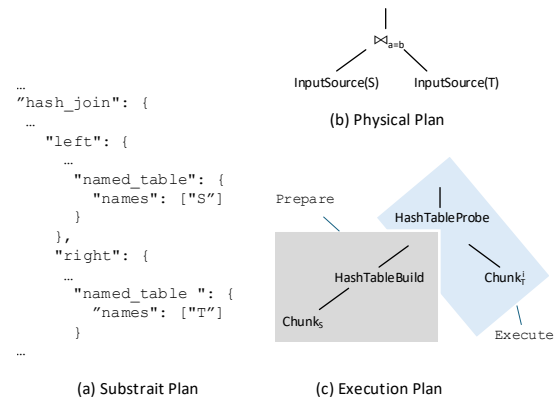


Figure 7: Life of a query in a GPU coprocessor.

previous section, whereby non-partitionable tables are registered and loaded first. In analytical engines, input data is often in columnar format, amenable to efficient compression using lightweight encoding. Input data chunks are sent to the coprocessor in their native format to save PCIe and memory bandwidth, and to delegate expensive decoding to the GPU. After inputs are sent and registered into the coprocessor, we Bind the Plan to the input chunks and Build an *Execution Plan* specific for that query fragment and input chunks (see Figure 7(c)). Input *Physical Plans* coming from Substrait only store “shallow” descriptions of the inputs, with no data (InputSource S and T in Figure 7(b)). The Bind steps swap those descriptions in the plan leaves with real chunks of data. Next, the Build process accomplishes the following. First, starting with the input chunks now stored in the leaves of the plan, it traverses the tree up to the next pipeline-breaker operation [55] such as hash table build (for Prepare calls) or final result generation (for Execute calls). This defines the query fragment sub-plan that will be executed by the Runtime. Second, a compilation stage is triggered where the *Physical Plan* is converted into an *Execution Plan* that can be run in a coprocessor (detailed below). The example of Figure 7(c) shows two *Execution Plans* coming from the same input *Physical Plan*: one for the Prepare (in gray) and one for Execute (in light blue), each of them executing over one or more chunks of data. *Execution Plans* consist of operators specifically picked based on input characteristics (e.g., hash tables built for 8-bit integer inputs).

Query Execution. Each *Operator* in an *Execution Plan* implements a simple interface containing a Run method that triggers the execution of the operator on GPU. *Execution Plans* are run in the usual pull-based mode of vectorized databases: calling Run on the root of the plan triggers a call to Run on each of the children, and this continues recursively until a leaf node operator wrapping a data chunk is reached, which then returns itself. In general, each *Operator* is column-based, in the sense that it receives as input one or more columns (chunks) and it produces as output one or more columns (chunks). When constructed during Build, each *Operator* is instantiated with a lambda function containing the actual logic. A dispatcher is used to pick an implementation for a specific *Operator* based on the input/output types and encodings. The evaluation of *Execution Plans* is triggered by Prepare API calls over non-partitionable tables or Execute calls over the partitionable table. In the former case, the output is an intermediate result, such as a hash table, which is cached in DAL and reused for each input chunk being used in Execute calls. In the latter case, the output is a final Resultset that is passed to the host for consumption (e.g., moved to the CPU or shuffled over the network to other coprocessors).

Dealing with Updates. Modern Data Lakes utilize table formats [8, 13, 75] for managing structured data, with Microsoft Fabric currently using Delta Parquet [8] as its underlying table format (and also supporting Iceberg via XTable [2] metadata translation). These formats inherently support update and delete operations on rows, necessitating that coprocessors accommodate data with modifications to maintain consistency with Snapshot Isolation transaction semantics [4]. To address this requirement, we support deletion vectors by introducing an additional boolean column linked to each table at Bind-time. This *deletion bitmap* serves as a mask that identifies invalid or deleted rows within the input data. At query time, the

coprocessor applies this bitmap as a filter. This approach ensures that updates and deletions are properly handled³.

3.4 GPU Coprocessor Implementation Details

TQP. The GPU coprocessor implementation started as a C++ port of TQP [32] with several improvements. As in TQP, the GPU coprocessor uses PyTorch as the underlying framework for data abstraction (i.e., we use the PyTorch’s tensor abstraction to represent relational columnar data [41]) as well as for the basic operations over data, and memory management (i.e., pooled allocator). As in TQP, we follow the sub-operator [37] strategy of exploding relational operator implementations into smaller reusable sub-operators. In contrast to TQP, we split the compilation phase into two: Substrait to *Physical Plan*, and *Physical Plan* to *Execution Plan*. This enables us to (1) cache the *Physical Plan* and reuse it across runtimes without having to re-generate it, and (2) optimize the *Physical Plan* for GPU before it is lowered into executable form. Each operator can have different implementations (TQP provided only one). An implementation is picked based on input data types and format (e.g., plain or encoded). This additional flexibility allows us to inject data into the coprocessor in its natural format (e.g., columnar and encoded) and enables efficient execution while minimizing data decoding [35]. While PyTorch operations were sufficient for research settings, in production we needed finer-grained control over the semantics of operations for: (1) handling overflows, (2) supporting nulls, and (3) maximizing performance. While we still rely on PyTorch operators, we re-implemented several of them in plain CUDA. We also implemented support for DAL to ensure optimal data movement and caching performance; we further detail this in Section 3.6. From now on, we will use TQP as a synonym for the GPU coprocessor.

Data Representation. Internally TQP uses PyTorch tensors as its data abstraction. This makes the representation of numerical columnar data (integers or floating point) natural as one-dimensional tensors, while decimals can be similarly represented as int64 2D tensors. Date-related types are also physically represented as integers (e.g., Datetime2 is represented as an 8-byte integer in Fabric DW, and Date is similarly represented as a 4-byte integer). We use Substrait’s type variations [74] and function overloading to pick the proper implementation at Build-time.⁴ Similarly, as we reuse sub-operators to implement relational operations, we try to use type variations and function overloading as much as possible to minimize the set of operations implemented in GPU. Dictionary-encoded strings are the most complex data type currently supported in TQP. We originally used a one-byte-padded 2D tensor representation for strings. Despite its simplicity, the memory requirements and performance makes this approach infeasible for production. To address this, we re-implemented dictionary-encoded strings as four 1D tensors: one for the string identifiers (tensor id) and three for the dictionary, which comprises (1) the concatenated set of n distinct strings, (2) the offsets of each string within the string tensor, and (3) the length of each string. The offset and length tensors have the same size, and the domain of the id tensor is $[0..n]$. The id tensor

³Updates in Delta Parquet and Iceberg are implemented as deletions+insertions, with the deletion vector tracking which historical rows should be excluded from results.

⁴For example, we use integer comparison operators for comparisons between date columns, while we use the proper function overload for casting or string conversions.

has one entry for each string in the (logical) column, and each entry is used to index in the offset tensor to fetch the string. This representation is built from Parquet and SQL Server columnar inputs with almost no overhead (beyond the CPU-to-GPU data copy).

Tensor Algebra Language. *Execution Operators* are instantiated with lambda functions containing the GPU implementation. The *Tensor Algebra Language (TAL) Operators* make the last layer of operators in TQP and are the implementations of the lambda functions used inside *Execution Operators*. *TAL Operators* are the sub-operators that compose the final computation graph that runs on the GPU. TQP comes with a fixed set of pre-compiled *TAL Operators* that are pre-loaded in TQP at startup time. TQP currently has about 150 TAL operators: some of them are wrappers around PyTorch operators (e.g., `tal_index_select`, `tal_masked_select`), others are custom CUDA kernels (e.g., `tal_hash_table_build`, `tal_topk_with_ties`). *TAL Operators* are templated based on the input data type and format (e.g., is the input column encoded, is it plain, does it contain nulls). The dispatcher determines which implementation to use at compile-time during Build, using type propagation to infer input types and formats from the bound plan leaf nodes. *TAL Operators* forming an *Execution Plan* can be dumped for offline analysis or profiling, either as a graph or in plain text. Text-based execution logs are represented in Static Single Assignment (SSA) form. Listing 1 shows a simplified text dump example for TPC-H Query 6. This is generated by a call to `Execute` (line 1). Each variable represents either a column (e.g., `L_SHIPDATE` in line 2), a constant (e.g., `constant_727928` in line 3), or the result of a TAL operator (e.g., line 4). Every variable definition specifies its type (e.g., `bool`) and arity (e.g., `95,894,053` for `L_SHIPDATE`, `1` for constants). The program flow is straightforward: a series of comparison and logical operations evaluate the filter predicate. Row indexes satisfying the predicate are produced by `tal_nonzero`, which returns the indexes of all non-zero elements in a column. These indexes are used to materialize row values (lines 21–22) and compute the final sum aggregate (line 25). The result is a `Resultset` containing a 16-byte decimal value (line 26).

Listing 1: Runtime TQP Execution log for TPC-H Query 6

```

1  {},SECTION,{Runtime_Gpu::DoExecute}
2  {[int32;L_SHIPDATE;95894053]},input
3  {[int32;constant_727928;1]},input
4  {[bool;result_37;95894053]},tal_ge,{L_SHIPDATE;constant_727928}
5  {[int32;constant_728293;1]},input,{ }
6  {[bool;result_38;95894053]},tal_lt,{L_SHIPDATE;constant_728293}
7  {[bool;result_39;95894053]},tal_logical_and,{result_37;result_38}
8  {[int8;L_DISCOUNT;95894053]},input
9  {[int8;constant_5_dec_2_3;1]},input
10 {[bool;result_40;95894053]},tal_ge,{L_DISCOUNT;constant_5_dec_2_3}
11 {[bool;result_41;95894053]},tal_logical_and,{result_39;result_40}
12 {[int8;constant_7_dec_2_3;1]},input
13 {[bool;result_42;95894053]},tal_le,{L_DISCOUNT;constant_7_dec_2_3}
14 {[bool;result_43;95894053]},tal_logical_and,{result_41;result_42}
15 {[int16;L_QUANTITY;95894053]},input
16 {[int16;constant_2400_dec_2_12;1]},input
17 {[bool;result_44;95894053]},tal_lt,{L_QUANTITY;constant_2400_dec_2_12}
18 {[bool;result_45;95894053]},tal_logical_and,{result_43;result_44}
19 {[int32;result_46;11421368]},tal_nonzero,{result_45}
20 {[int32;L_EXTENDEDPRICE;1]},input
21 {[int32;result_47;11421368]},tal_index_select,{L_EXTENDEDPRICE;result_46}
22 {[int8;result_48;11421368]},tal_index_select,{L_DISCOUNT;result_46}
23 {[int64;result_49;11421368]},tal_mul,{result_47;result_48}
24 {[int64;result_50;11421368]},tal_shallow_copy,{result_49}
25 {[int128;result_51;1]},tal_agg_sum,{result_50}
26 {},Resultset_Gpu_1x1_16_b,{ }
27 {},SECTION,{Runtime_Gpu::DoExecute_end}

```

Intrinsics Support. Fabric DW exposes hundreds of intrinsic operations such as `LIKE`, `YEAR`, and `SUBSTRING`. Re-implementing them in CUDA is a major engineering undertaking, but necessary to fully accelerate queries in GPU. Otherwise, sending data back and forth to the PCIe bus might incur an overhead that makes end-to-end execution slower than just running the query on the host. To overcome this problem, we developed a compiler that directly takes C++ code from the Fabric DW codebase and translates it into PTX executable on the GPU. While the process is currently not yet fully automated, we have ported a few dozen intrinsics from Fabric DW to TQP, with performance that sometimes is even better than hand optimized code. We refer interested readers to [66] for more details.

3.5 Beyond GPU coprocessors

The complexity and cost of our shift to accelerated analytics run deep. A core principle was avoiding bets with single points of failure. Our early research [9, 29] showed promise for GPUs, but we still enforced hardware and vendor independence at every layer. We also explored other paths: an in-house FPGA prototype and a partnership with the custom-ASIC startup Speedata [73]. These efforts kept us honest about hardware flexibility and demonstrated the viability of alternative platforms. The CAL design lets us dispatch parts of a query to diverse coprocessors in a cost-based manner. Next, we summarize our experience with non-GPU coprocessors.

FPGA. We leveraged an FPGA solution built on the Intel Agilex 7 FPGA I-Series AGI 041. The FPGA implements the same CAL API as the GPU coprocessor, which allows the FPGA to fetch SQL Server's internal columnar data directly from host memory using application-level DMA, enabling streaming-based processing and conserving CPU cycles. The data path implemented on the FPGA includes modules for *DMA*, *scan*, *data decoding*, *filtering*, *partitioning*, and *hash-join* operations, orchestrated by a *Memory Object Manager*. To accelerate development productivity, we use Kanagawa [63], a vendor-agnostic high-level synthesis language and toolchain developed by Microsoft. Our implementation includes a runtime layer called *FPGA Manager* that bridges the CAL API and the low-level accelerator APIs, managing control and data paths.

Speedata ASIC. Starting in late 2023, we engaged with Speedata [73], a startup building an ASIC for data processing called Analytic Processing Unit (APU). Speedata's C200 APU is a CGRA-based dataflow machine on a 16 lane PCIe 5.0 board with 32GB of onboard RAM for processing. The board is capable of zero-copying columnar data files (e.g., Parquet) directly from host memory, performing data decompression and decoding in hardware, as well as relational operators such as select, project, join, and aggregation. The APU supports multiple data types including complex/nested structures and can process them in streaming mode (e.g., string manipulation, list explode etc.) and thus supporting all core SQL primitives. The APU operates at PCIe 5.0 line-rate for all one-pass operations. In the past year, we worked closely with Speedata to integrate their APU as an alternative implementation of the CAL API.

Discussion. While we are still in early evaluation stages of the FPGA and APU technology, both have been invaluable to demonstrate our ability to adapt to alternative and custom hardware, and explore solutions with different price-performance trade-offs when compared with the powerful but costly and power-hungry GPUs.

3.6 Data Movement and Accelerated Networks

We next cover DAL's *caching* and *distributed data shuffle*, which are surfaced through a *unified API* (see [50] for more details).

Caching and Memory Management. Modern GPUs come with 80-192 GB of HBM memory, with the expectation that next generation GPUs will have over 400 GB [6]. This is a non-trivial amount of memory. DAL leverages this large pool of memory both for storing intermediate results coming from the coprocessors (e.g., hash tables built from non-partitionable tables) and caching inputs (in their native format). Currently, the GPU cache implementation follows an LFRU strategy [42] and evicts entries when there is memory pressure. DAL additionally caches GPU memory allocations to avoid runtime overheads. Both DAL and TQP currently utilize the PyTorch pooled allocator as their unified memory manager. This supports performant allocations while also giving us the flexibility to implement cache eviction through callbacks.

Data Shuffle. In distributed settings, moving data across the network (shuffle) can consume large portions of the total end-to-end time. To address this bottleneck, DAL is designed to leverage any available interconnect provided by the host infrastructure. Specifically, DAL supports both high-performance interconnects such as NVLink and InfiniBand [58, 59], as well as simple PCIe and Ethernet. In our current implementation, we manage interconnects through UCX [70], an open-source communication framework that offers fine-grained control over how endpoints (e.g., NICs or GPUs) communicate. Our system establishes a hierarchy of preferred network transports: two endpoints prefer to communicate via NVLink if available, followed by InfiniBand, PCIe, and finally, Ethernet. DAL shuffle implementation employs a local-write/remote-read strategy.

DAL API. Shuffle and caching components expose a key/value API where both keys and values are opaque binary values. This choice allows coprocessors to freely represent references to data as well as data types. For local reads, DAL enables zero-copy data movement when possible. For remote fetch of data, DAL provides a GET API which assumes that callers know the location from where the data must be fetched. In the case of shuffle, these location specifiers are part of the input information encoded in the Substrait plan. Under this approach, the local DAL service does not need to maintain knowledge of where data resides. Instead, this information is managed in the host scheduler (e.g., Polaris [3] for Fabric DW).

4 Experimental Evaluation

We evaluate GPU-coprocessor-enabled Fabric DW on a single backend (Section 4.1) and distributed (Section 4.2) across various TPC-H scales and customer workloads. We also explore various caching approaches and networking technologies for DAL, and FPGA and ASICs for CAL (Section 4.3). Our experiments show consistent performance gains over CPU-only baselines, with increased benefits for larger datasets and complex analytical queries. In the following, we will use CoddSpeed for Fabric DW with the TQP implementation of CAL. We will be varying the DAL implementation/configuration.

4.1 Single CoddSpeed Backend

We evaluate a single backend version for CoddSpeed. We run the CPU baselines on an AMD EPYC 7V13 (Milan) processor with 24

cores and 220 GB of RAM. We report end-to-end speedups for CoddSpeed normalized over Fabric DW on CPU, computed over the median of 5 runs. For CoddSpeed, we vary DAL configuration and report *warm* numbers where the datasets are already cached in GPU memory, as well as *cold* numbers where the dataset is in CPU memory and is moved to the GPU during execution.

4.1.1 TPC-H Scale Factor 100. The hardware we use in this first set of experiments is a Standard NC24ads A100 v4 VM with an NVIDIA A100 GPU with 80 GB of HBM memory. The results are in Figure 8. As a reference, for this experiment we also plot the speedup over the same hardware, queries, and dataset for DuckDB (version 1.3.2) [65] and HeavyDB (version 8.5 EE) [33], a state-of-the-art commercial GPU database. In aggregate (last column of the plot) we can see that CoddSpeed is able to provide speedups over baseline CPU Fabric DW from $7.9\times$ (*warm*) to $4.7\times$ (*cold*). In comparison, HeavyDB with data warm in GPU is just $3\times$ faster⁵, while DuckDB is 20% better. If we look more in detail at each query, CoddSpeed with warm cache is always faster than the baseline, but the speedup varies from few percentage for scan-heavy queries like Q06 and Q19, to $33\times$ for Q13. In general, the larger speedup is coming from the more computationally heavy queries (e.g., Q09, Q13, Q18). Interestingly, HeavyDB is faster than CoddSpeed for Q03, Q05 and Q12 (in the latter by more than $2\times$). This is because (1) optimized kernels in HeavyDB, (2) different query plans generated by Calcite, and (3) overheads of the CAL integration. The cost of moving data from CPU to GPU is not negligible, and in fact we see that CoddSpeed cold is slower than baseline Fabric DW in scan heavy queries (e.g., Q01, Q06, Q19) while still outperforming it for the complex ones (e.g., by up to $30\times$ for Q13). Figure 12 illustrates the progression of speedup improvements leading up to private preview. While overall performance increased steadily, occasional dips occurred. These dips typically coincided with the introduction of new functionalities (e.g., overflow handling), which reduced performance until subsequent optimizations enhanced efficiency.

4.1.2 Internal Production Workload Scale Factor 300. In this experiment we evaluate how the GPU coprocessor behaves on production workloads. Differently from synthetically generated data in TPC-H, production workloads over real world data shows different patterns and data is often more compressible. We use 16 queries coming from an internal customer, all running over a 300GB (compressed) real-world dataset. Some of the queries are considerably more complex than TPC-H, e.g., we have queries with 10 joins, aggregation, and several intrinsic operations. For this experiment we use the same A100 GPU configuration as described in Section 4.1.1. The results are shown in Figure 13. In this case, we have an aggregate speedup of $14.9\times$ (*warm*) and $9.7\times$ (*cold*) compared to baseline Fabric DW on CPU. This proves that CoddSpeed can provide even better speedups for real workload workloads with more compressed data and more complex query patterns. Nevertheless, the speedup is largely coming from the more complex queries (Q03, Q08, Q12).

4.1.3 Upgrading the GPU. In this experiment we study the performance benefit of upgrading the GPU from the A100 used in the previous experiments to an H100. Specifically, we use a Standard

⁵We omit Q21 for HeavyDB, which is unsupported due to Calcite [11] inability to decorrelate the query.

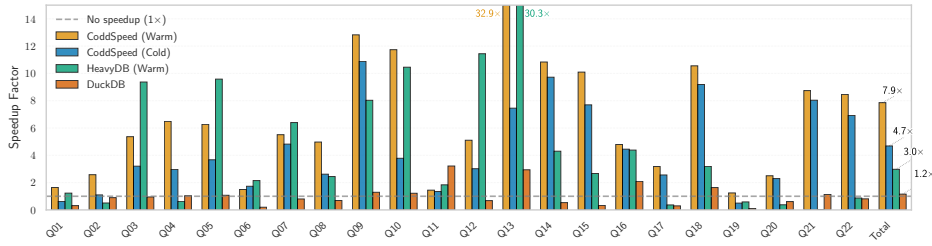


Figure 8: Speedup of CoddSpeed, HeavyDB, DuckDB, and baseline CPU Fabric DW for TPC-H SF 100. HeavyDB executes with data warm in GPU memory.

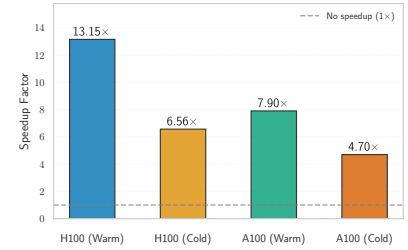


Figure 10: CoddSpeed with H100 and A100 vs. CPU for TPC-H SF 100.

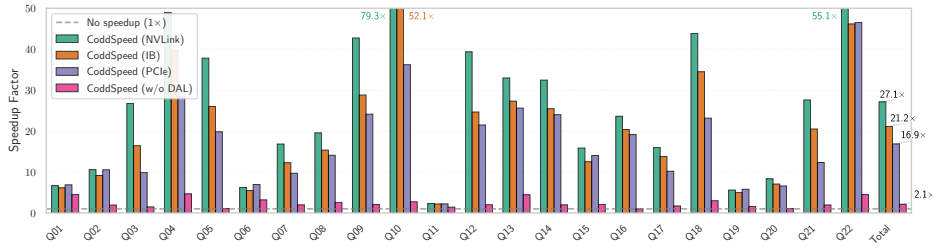


Figure 9: Speedup between CoddSpeed using 8 backends with GPU coprocessor vs. baseline Fabric DW on CPU (with up to 40 backends) for TPC-H SF 1000. Networking stack varied (NVLink, InfiniBand (IB), PCIe).

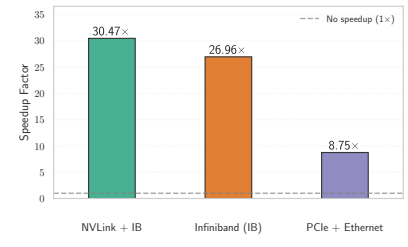


Figure 11: Speedup over Fabric DW for TPC-H SF 1000 using 16 GPUs and different network technologies.

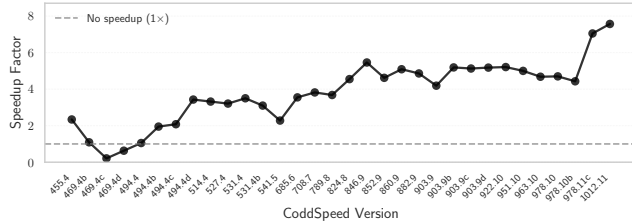


Figure 12: Improvement for TPC-H 100GB of single GPU CoddSpeed over time and leading to private preview.

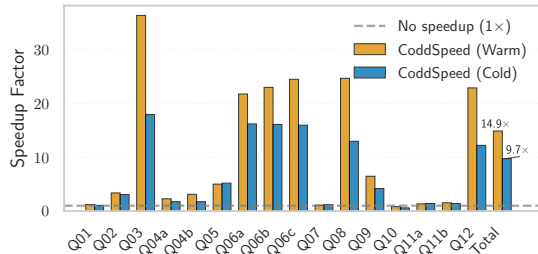


Figure 13: Speedup comparison between CoddSpeed and baseline Fabric DW for the internal real-world workload with warm and cold GPU cache.

NCads H100 v5 VM with an 80GB H100 GPU and compare against the Fabric DW baseline numbers over the 24 cores Milan processor. The numbers are shown in Figure 10 (where for better readability we also ported the A100 numbers from Figure 8). We can see that by just upgrading the GPU we can improve the *warm* cache speedup over Fabric DW by 70% and the *cold* cache speedup by almost 40%. The improvement over the *warm* A100 numbers matches the theoretical expectation as H100 has about 70% better memory bandwidth than A100 (3.5TB/s vs. 2.1TB/s), but not the speedup in

the *cold* case because we double the bandwidth by moving from a PCIe-4 to a PCIe-5 device. This is because currently we are not at optimal PCIe utilization due to the input data format that requires to move many small pieces of data from the CPU to the GPU.

4.1.4 Discussion. CoddSpeed *warm* is the fastest GPU query processor system we are aware of. As illustrated in Figure 8, we are about 2x faster than HeavyDB, a commercial GPU database system. But following our internal benchmarks, we are also faster than state-of-the-art research prototypes. For instance, our H100 numbers are 10% faster than the state-of-the-art research prototype, SiriusDB [80] on GB200 (the new generation GPU architecture).⁶

4.2 Distributed CoddSpeed

We next show the distributed performance of CoddSpeed using DAL with different networking layers. We use a Virtual Machine Scale Set with Standard ND96isrf H100 v5 VMs each equipped with 8 H100 with 80GB of memory, 8 ConnectX7 with 400Gb/s of bandwidth, and one 100Gb/s Ethernet NIC. Each VM is equipped with NVLink and NVSwitch allowing collocated GPUs to communicate with each other at 450GB/s in each direction, for a total aggregate local bandwidth of 3.6TB/s. The nodes are connected through an InfiniBand (IB) backend network. We also use a Standard D4s v3 (4 vcpus, 16 GiB memory) VM as a frontend node to submit queries to the backend nodes. As a baseline, we use Fabric DW over Ethernet, and let Fabric DW decide how many nodes to use to execute each query. We evaluate TPC-H SF 1000 and show the end-to-end speedup of CoddSpeed over a single node with 8 GPUs (Section 4.2.1), and scalability results over two nodes (Section 4.2.2). We report the median of three runs with warm GPU caches.

⁶As of this writing, SiriusDB cannot execute TPC-H 100GB on an H100 because it requires the full uncompressed dataset to fit into GPU memory.

4.2.1 Single Node Multi-GPUs Performance. In Figure 9 we show speedup over the baseline Fabric DW with up to 8 GPUs/coprocessor on a single node. We show numbers without DAL and using DAL with NVLink/IB/PCIe as the networking layer. Switching networking layer is only a configuration change in DAL/UCX. If we first look at the totals, with TQP enabled, when we don't use DAL, we see a 2.1× improvement over the CPU baseline. Conversely, the speedups with both TQP and DAL enabled go from 27.1× with NVLink, to 21.2× with IB, to 16.9× with communication through PCIe. Without DAL, we are using the native Fabric DW shuffle implementation, which requires to (1) pull data from each coprocessor, (2) convert it from columnar to row format, (3) do partitioning in CPU, (4) send the data remotely, and (5) re-converting the data from row to columnar format. With DAL enabled, we can instead efficiently zero-copy data when shuffling. This approach, coupled with faster networking, is required to enable large improvements over the CPU baseline, as highlighted in our design principles in Section 2. Interestingly, there is quite a big gap between IB and PCIe, although both should have the same bandwidth (50 GB/s). Our conclusion is that: (1) with IB we can RDMA directly from GPUs' memory therefore avoiding a copy to CPU memory, and (2) each GPU is directly connected to a local NIC, therefore decreasing the pressure on the PCIe root system. If we look at how speedups are distributed over the queries, as noticed for the single GPU case, simpler queries in general have smaller speedups, while more complex queries show larger speedups. In the multi-GPU case, we see speedups of almost 80× and 55× with TQP and DAL with NVLink for queries Q10 and Q22, respectively.

4.2.2 Multi-Node Multi-GPUs Performance. We use up to 16 GPUs spread over 2 nodes. Figure 11 shows the speedups of CoddSpeed using three different networking stacks behind DAL vs. the CPU baseline. When using two times more GPUs, we get a top speedup of 30.4× with DAL backed by NVLink+IB.⁷ When only enabling IB, the speedup goes from 21.2× in the single node case, to 26.9× for the two node cases. Conversely, if we force DAL to use PCIe locally and the slow Ethernet across nodes, we are only seeing an 8.7× speedup, down from 16.9× in the single node PCIe case. We internally have run experiments over 24 and 32 GPUs, but the performance gains over the 16 GPUs case are negligible. This is because: (1) scheduling overheads in Polaris increase with the number of GPUs, and (2) the performance benefits of using more GPUs decrease as we add more. We defer more details to a separate publication [49].

4.3 Other Coprocessors

We next explore how hardware other than GPUs can bring acceleration to query processing. We share two set of experiments: (1) an end-to-end performance of SQL Server with the FPGA coprocessor (Section 4.3.1); and (2) raw performance (no host engine) of the Speeddata APU implementing the CAL API (Section 4.3.2). We compare both scenarios against the single core performance of SQL Server on an Intel Xeon Platinum 8473C CPU @ 2.80GHz, using the TPC-H dataset with scale factor 100.

⁷We use 16 GPUs on 2 nodes: shuffles local within a node use NVLink and those across remote nodes use IB. Similar configuration applies for PCIe (local) + Ethernet (remote).

4.3.1 SQL Server with FPGA. We use an Intel 8370C CPU with 32 cores connected to an Intel Agilex 7 FPGA I-Series AGI 041, featuring approximately 4 million logic elements, over 1.3 million adaptive logic modules, and 377 Mb of embedded memory. The hardware platform leverages PCIe Gen5 with dual x8 links, delivering up to 12 GB/s bidirectional bandwidth, and supports four DDR4 channels at 2666 MT/s for a peak memory bandwidth of 85.3 GB/sec. Through CAL, the FPGA streams data directly from SQL Server buffer pools through a pinned 4GB reusable buffer, decodes the SQL Server internal columnar format into its internal representation, executes the query, and returns the results back to SQL Server. Table 1 reports FPGA speedups over single core performance of SQL Server for TPC-H queries Q06, Q12, Q14, and Q19 at scale factor 100. Results show that there is quite a large performance difference across queries. For instance, the performance varies from 4.5× for Q06 to 48.1× for query Q14.⁸ This large difference is due to the computational complexity of the query. For example, Q6 has a simple filter and aggregation, while Q14 has compute-heavy joins and LIKE predicates.

Coprocessor	Q06	Q12	Q14	Q19
SQL Server + FPGA	4.5×	14.5×	48.1×	6.6×

Table 1: Speedup of the FPGA prototype vs. SQL Server single core over a subset of TPC-H queries (SF 100).

4.3.2 Speeddata APU. We report standalone performance between the time CAL is called and Resultsets are generated. We use TPC-H scale factor 100 in Parquet format, warm in CPU memory. Table 2 shows results that the prototype integrating the API with CAL can deliver large performance improvements, up to 259× for Q13 compared to single core SQL Server. This is because these queries are compute-heavy (e.g., Q13 has a LIKE predicate over the O_COMMENT column) and are largely amenable to hardware acceleration.

Coprocessor	Q07	Q10	Q12	Q13	Q16
Standalone APU	36.7×	40.7×	49.4×	259.2×	61.5×

Table 2: Speedup of the APU integration with CAL vs. SQL Server single core over a subset of TPC-H queries (SF 100).

5 Lessons Learned and Challenges

Next we share few key insights from our experience of building and integrating CAL and DAL on different hosts and hardware.

Incremental Adoption. Transitioning to hardware-accelerated query processing should be evolutionary, not revolutionary. Rather than replacing entire systems, our approach is to start by integrating coprocessors into existing architectures and iteratively increase the surface area. This minimizes risk, leverages current investments and customers, and allows us to validate improvements incrementally.

Pushdown Granularity. Pushing only a few operators to the coprocessor rarely delivers meaningful speedups since transfer costs often outweighs an accelerator's benefits. A lesson we learned early on is that to achieve true acceleration we must push down large query fragments and minimize back-and-forth data movements between the accelerator and CPU.

⁸The prototype uses half of the resources on the FPGA. We expect that we could decrease FPGA run times when we use all the resources on the FPGA board.

Abstracting Hardware and Network for Future-Proof Design.

One of the key lessons learned from TQP [32, 79] is that with the proper abstractions in place, AI infrastructure can be leveraged to perform database analytics. When transitioning TQP from research to product, we followed the same philosophy by decoupling the engine from specific hardware (GPU, FPGA, ASIC) and network stacks (InfiniBand, NVLink, Ethernet). This ensures portability, resilience to resource scarcity, and future-proofing, while still allowing hardware-specific optimizations behind the abstractions.

PyTorch Lift. Using PyTorch initially accelerated development due to its rich ecosystem and familiarity. As performance and fine-grained control became critical, migrating to custom implementations (e.g., specialized kernels or hardware APIs) was essential, which unlocked optimizations like operator fusion and execution over compressed data.

6 Related Work

In the past decade, the field of GPU-accelerated database systems has seen significant research activity. Academic systems such as Ocelot [34], YDB [76, 83], Crystal [71] and others [1, 15, 16, 20, 21, 31, 38, 43, 68, 72, 81] have explored various approaches to GPU acceleration. Commercial solutions including RAPIDS cuDF [67], BlazingSQL [12], HeavyDB [33], Kinetica [40], and Theseus [7] have demonstrated the viability of GPU acceleration in production environments. Concurrently, there has been substantial research on accelerating relational operators and analytics queries on FPGAs [24, 25, 27, 28, 47, 51, 60, 77, 78, 84], as well as commercial solutions, including Netezza [54], AQUA [10], and others [64]. Our work differs from prior approaches in some key aspects. First, our hardware-agnostic API represents the first future-proof, cloud-scale commercial approach for accelerated analytics able to span different hardware coprocessors and networking technology. Our partitioned table approach and Substrait specifications allow us to easily integrate and deploy a coprocessor within existing enterprise data warehouse architectures with minimal modifications.

Recent efforts [26] such as the Composable Data Management System Manifesto [62] highlight a growing trend toward decomposing monolithic DBMS architectures into modular, reusable components. Velox [61] introduces a unified query execution engine for heterogeneous environments, primarily CPU-based with optional GPU acceleration [39]. While Velox emphasizes extensibility and operator-level optimizations, our approach introduces a hardware-agnostic API spanning GPUs, FPGAs, and networking accelerators, enabling cloud-scale deployments.

The work presented in this paper has been seeded by a multi-year research effort around TQP [9, 14, 18, 23, 29, 32, 35, 41, 53, 79]. Beyond the prototypes already discussed in the paper, there are other extensions we are currently considering for integration:

Hybrid Execution. Our current implementation allows splitting query plans across CPU and GPU where each split is executed sequentially. Alternatively, we investigated an extension that explored CPU-GPU co-execution [44]. For large datasets that do not fit in GPU memory, we can execute filters in CPU memory in compressed form, and then transfer the (still compressed) results to the GPU to execute the compute-heavy operations. This strategy

enables running TPC-H SF 1000 on a single A100 GPU with up to 4× performance improvement compared to SQL Server on CPU.

Operator Fusion. Historically, operator fusion and compilation techniques were proposed to improve query processors' efficiency [55]. For TQP, this technique is critical since each sub-operator materializes its results, increasing memory pressure. Although we explored operator fusion and compilation in research settings with excellent results [23], performing runtime compilation in a GPU engine is challenging since generating PTX code takes hundreds of milliseconds. By contrast, our approach is to persist *Execution Plans* and generate fused TAL operations offline. These fused operations are subsequently reused by TQP, allowing it to benefit from operator fusion without having the pay compilation costs at runtime.

Execution over Encoded Data. Data compression is a well-known technique for reducing the overhead of data movement over low-bandwidth interconnects such as the PCIe bus. In TQP, we extend this idea by operating directly on compressed data where possible, thereby freeing up HBM capacity while also improving performance. This is in contrast to traditional approaches of decoding data in the GPU memory hierarchy before operating on it [82]. Our research [35] has shown how relational operators can be implemented directly on encoded data resulting in an order-of-magnitude speedup over commercial CPU RDBMS and analytics engines for processing representative real-world queries on production datasets. We are continuing on this journey and enhancing our capabilities in two directions: (1) operating on bit-packed in addition to dictionary and RLE data represented in SQL Server internal columnar data format, and (2) enabling shuffle and broadcast operations to work with encoded data during distributed query execution.

7 Conclusions

In this work, we presented our experience integrating coprocessors into an enterprise-scale data warehouse system. Our Coprocessor Abstraction Layer and partitionable execution model enable seamless integration across diverse deployment scenarios. Our results show that coprocessors can deliver integer factor performance improvements on single-node execution. However, at scale, data movement becomes the primary bottleneck. To address this, we further introduced a Data Abstraction Layer that hides the network transport medium, achieving up to 30× speedup over CPU-only solutions. Through proper abstraction, we demonstrate that Data Warehouse systems can be future-proofed to leverage current and next-generation accelerated compute and networking technologies.

8 Acknowledgements

We thank engineers, program managers, and researchers at Microsoft: Abhijeet Lawande, Austin Baylis, David Brookler, Dharini Sundaram, Gabriel Spil, John Demme, Junwei Wei, Kara Kagi, Krystian Zrodowski, Ljubica Vujovic Boskovic, Nevena Nikolic, Parker Mitchell, Raja Chowdhury, Rajkumar Iyer, Santhosh Kumar Ravindran, Sowmya Sivaraman, Sricheta Ruj, Tino Tereshko, Todd Mas-sengill; students and professors: Bowen Wu, Dong He, Gustavo Alonso, Jiashen Cao, Matthias Jasny, Spyros Blanas, Supun Nakan-dala, Zezhou Huang, and collaborators in NVIDIA and Speedata: Adi Gelvan, Boaz Menuhin, Dan Eaton, Itai Incze, Jonathan Friedmann, Matan Peleg, Ori Netzer, Tom Deane.

References

- [1] Azim Afrozeh, Lotte Felius, and Peter Boncz. 2024. Accelerating GPU Data Processing using FastLanes Compression. In *Proceedings of the 20th Workshop on Data Management on New Hardware*. ACM, Santiago, Chile, 1–11.
- [2] Ashvin Agrawal, Tim Brown, Anoop Johnson, Jesús Camacho-Rodríguez, Kyle Weller, Carlo Curino, and Raghu Ramakrishnan. 2024. XTable in Action: Seamless Interoperability in Data Lakes. *CoRR* abs/2401.09621 (2024), 1–4. <https://doi.org/10.48550/ARXIV.2401.09621> arXiv:2401.09621
- [3] Josep Aguilar-Saborit et al. 2020. POLARIS: The Distributed SQL Engine in Azure Synapse. *Proc. VLDB Endow.* 13, 12 (2020), 3204–3216. <https://doi.org/10.14778/3415478.3415545>
- [4] Josep Aguilar-Saborit et al. 2024. Extending Polaris to Support Transactions. In *Companion of the 2024 International Conference on Management of Data, SIGMOD/PODS*, Pablo Barceló, Nayat Sánchez-Pi, Alexandra Meliou, and S. Sudarshan (Eds.). ACM, Santiago, Chile, 321–333. <https://doi.org/10.1145/3626246.3653392>
- [5] AMD. 2025. AMD Infinity Fabric Link. <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/other/56978.pdf>.
- [6] AMD. 2025. AMD Previews 432 GB HBM4 Instinct MI400 GPUs and Helios Rack-Scale AI Solution. https://www.techpowerup.com/337987/amd-previews-432-gb-hbm4-instinct-mi400-gpus-and-helios-rack-scale-ai-solution?utm_source=chatgpt.com.
- [7] Felipe Aramburú et al. 2025. Theseus: A Distributed and Scalable GPU-Accelerated Query Processing Platform Optimized for Efficient Data Movement. arXiv:2508.05029 [cs.DC] <https://arxiv.org/abs/2508.05029>
- [8] Michael Armbrust, Tathagata Das, Sameer Paranjpye, Reynold Xin, Shixiong Zhu, Ali Ghodsi, Burak Yavuz, Mukul Murthy, Joseph Torres, Liwen Sun, Peter Boncz, Mostafa Mokhtar, Herman Van Hovell, Adrian Ionescu, Alicja Luszczak, Michal Swiatkowski, Takuya Ueshin, Xiao Li, Michal Szafranski, Pieter Senster, and Matei Zaharia. 2020. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. In *Proc. VLDB Endowment*, Vol. 13. VLDB Endowment, Tokyo, Japan, 3411–3424.
- [9] Yuki Asada, Victor Fu, Apurva Gandhi, Advitya Gemawat, Lihao Zhang, Dong He, Vivek Gupta, Ehi Nosakhare, Dalitso Banda, Rathijit Sen, and Matteo Interlandi. 2022. Share the Tensor Tea: How Databases can Leverage the Machine Learning Ecosystem. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3598–3601. <https://doi.org/10.14778/3554821.3554853>
- [10] Jeff Barr. 2021. AQUA (Advanced Query Accelerator) – A Speed Boost for Your Amazon Redshift Queries. [Online] Available from: <https://aws.amazon.com/blogs/aws/new-aqua-advanced-query-accelerator-for-amazon-redshift/>.
- [11] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J. Mior, and Daniel Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, Houston, TX, 221–230. <https://doi.org/10.1145/3183713.3190662>
- [12] BlazingSQL. (last accessed) 2025. BlazingSQL. [Online] Available from: <https://github.com/BlazingDB/blazingsql>.
- [13] Ryan Blue, Kostas Tzoumas, et al. 2020. Apache Iceberg: A Table Format for Huge Analytic Datasets. <https://iceberg.apache.org/>.
- [14] Matthias Boehm, Matteo Interlandi, and Chris Jermaine. 2023. Optimizing Tensor Computations: From Applications to Compilation and Runtime Techniques. In *Companion of the 2023 International Conference on Management of Data, SIGMOD/PODS 2023, Seattle, WA, USA, June 18-23, 2023*, Sudipto Das, Ippokratis Pandis, K. Selçuk Candan, and Sihem Amer-Yahia (Eds.). ACM, Seattle, 53–59. <https://doi.org/10.1145/3555041.3589407>
- [15] Nils Boesch, Tobias Ziegler, and Carsten Binnig. 2024. GOLAP: A GPU-in-Data-Path Architecture for High-Speed OLAP. *Proceedings of the ACM on Management of Data* 2, 6, Article 237 (Dec. 2024), 26 pages. <https://doi.org/10.1145/3698812>
- [16] Sebastian Breß. 2014. The Design and Implementation of CoGaDB: A Column-oriented GPU-accelerated DBMS. *Datenbank-Spektrum* 14, 3 (2014), 199–209.
- [17] Nicolas Bruno, César A. Galindo-Legaria, Milind Joshi, Esteban Calvo Vargas, Kabita Mahapatra, Sharon Ravindran, Guoheng Chen, Ernesto Cervantes Juárez, and BeySim Sezgin. 2024. Unified Query Optimization in the Fabric Data Warehouse. In *Companion of the 2024 International Conference on Management of Data, SIGMOD/PODS 2024, Santiago, Chile, June 9-15, 2024*. ACM, <https://doi.org/10.1145/3626246.3653369>, 18–30.
- [18] Jiashen Cao, Rathijit Sen, Hyesoon Kim, and Matteo Interlandi. 2024. GPU Database Systems Characterization and Optimization. *Proceedings of the VLDB Endowment* 17, 3 (2024), 441–454. PDF available.
- [19] Cerebras. 2025. The fastest AI infrastructure. <https://cerebras.ai/>.
- [20] Periklis Chrysogelos, Manos Karpathiotakis, Raja Appuswamy, and Anastasia Ailamaki. 2019. HetExchange: encapsulating heterogeneous CPU-GPU parallelism in JIT compiled engines. *Proc. VLDB Endow.* 12, 5 (Jan. 2019), 544–556. <https://doi.org/10.14778/3303753.3303760>
- [21] Periklis Chrysogelos, Panagiotis Sioulas, and Anastasia Ailamaki. 2019. Hardware-conscious query processing in GPU-accelerated analytical engines. *Memory* 1 (2019), L2.
- [22] Compute Express Link. 2025. CXL. <https://computeexpresslink.org/about-cxl/>.
- [23] Wei Cui, Qianxi Zhang, Spyros Blanas, Jesús Camacho-Rodríguez, Brandon Haynes, Yinan Li, Ravi Ramamurthy, Peng Cheng, Rathijit Sen, and Matteo Interlandi. 2023. Query Processing on Gaming Consoles. In *Proceedings of the 19th International Workshop on Data Management on New Hardware, DaMoN 2023, Seattle, WA, USA, June 18-23, 2023*, Norman May and Nesime Tatbul (Eds.). ACM, Seattle, 86–88. <https://doi.org/10.1145/3592980.3595313>
- [24] Christopher Denny, Daniel Ziener, and Jürgen Teich. 2012. On-the-fly Composition of FPGA-Based SQL Query Accelerators Using a Partially Reconfigurable Module Library. In *2012 IEEE 20th International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, Toronto, Canada, 45–52. <https://doi.org/10.1109/FCCM.2012.18>
- [25] Christopher Denny, Daniel Ziener, and Jürgen Teich. 2013. Acceleration of SQL Restrictions and Aggregations through FPGA-Based Dynamic Partial Reconfiguration. In *2013 IEEE 21st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, Seattle, Washington, 25–28. <https://doi.org/10.1109/FCCM.2013.38>
- [26] VLDB Endowment. 2025. Proceedings of the Third International Workshop on Composable Data Management Systems. <https://cdmsworkshop.github.io/2025/>
- [27] Zubeyr F. Eryilmaz, Aarati Kakaraparthi, Jignesh M. Patel, Rathijit Sen, and Kwanghyun Park. 2021. FPGA for Aggregate Processing: The Good, The Bad, and The Ugly. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, Chania, Greece, 1044–1055. <https://doi.org/10.1109/ICDE51399.2021.00095>
- [28] Jian Fang, Yvo T. B. Mulder, Jan Hidders, Jinho Lee, and H. Peter Hofstee. 2019. In-memory database acceleration on FPGAs: a survey. *The VLDB Journal* 29, 1 (Oct. 2019), 33–59. <https://doi.org/10.1007/s00778-019-00581-w>
- [29] Apurva Gandhi, Yuki Asada, Victor Fu, Advitya Gemawat, Lihao Zhang, Rathijit Sen, Carlo Curino, Jesús Camacho-Rodríguez, and Matteo Interlandi. 2023. The Tensor Data Platform: Towards an AI-centric Database System. In *13th Conference on Innovative Data Systems Research, CIDR 2023, Amsterdam, The Netherlands, January 8-11, 2023*. www.cidrdb.org/cidr2023/papers/p68-gandhi.pdf
- [30] Goetz Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data Eng. Bull.* 18, 3 (1995), 19–29. <http://sites.computer.org/debull/95SEP-CD.pdf>
- [31] Bingsheng He, Mian Lu, Ke Yang, Rui Fang, Naga K. Govindaraju, Qiong Luo, and Pedro V. Sander. 2009. Relational query coprocessing on graphics processors. *ACM Transactions on Database Systems (TODS)* 34, 4, Article 21 (Dec. 2009), 39 pages. <https://doi.org/10.1145/1620585.1620588>
- [32] Dong He, Supun Chathuranga Nakandala, Dalitso Banda, Rathijit Sen, Karla Saur, Kwanghyun Park, Carlo Curino, Jesús Camacho-Rodríguez, Konstantinos Karanasos, and Matteo Interlandi. 2022. Query Processing on Tensor Computation Runtimes. *Proc. VLDB Endow.* 15, 11 (2022), 2811–2825. <https://doi.org/10.14778/3551793.3551833>
- [33] Heavy.ai. (last accessed) 2025. HeavyDB. [Online] Available from: <https://www.heavy.ai/product/heavydb>.
- [34] Max Heime, Michael Saecker, Holger Pirk, Stefan Manegold, and Volker Markl. 2013. Hardware-oblivious parallelism for in-memory column-stores. *Proc. VLDB Endow.* 6, 9 (July 2013), 709–720. <https://doi.org/10.14778/2536360.2536370>
- [35] Zezhou Huang, Krystian Sakowski, Hans Lehnert, Wei Cui, Carlo Curino, Matteo Interlandi, Marius Dumitru, and Rathijit Sen. 2025. GPU Acceleration of SQL Analytics on Compressed Data. *CoRR* abs/2506.10092 (2025). <https://doi.org/10.48550/ARXIV.2506.10092> arXiv:2506.10092
- [36] Intel. 2025. Intel Demonstrates First Fully Integrated Optical I/O Chiplet. <https://www.intel.com/news-events/press-releases/detail/1699/intel-demonstrates-first-fully-integrated-optical-io>.
- [37] Michael Jungmair and Jana Giceva. 2023. Declarative Sub-Operators for Universal Data Processing. *Proc. VLDB Endow.* 16, 11 (2023), 3461–3474. <https://doi.org/10.14778/3611479.3611539>
- [38] Marko Kabić, Shriram Chandran, and Gustavo Alonso. 2025. Maximus: A Modular Accelerated Query Engine for Data Analytics on Heterogeneous Systems. *Proceedings of the ACM on Management of Data* 3, 3, Article 187 (June 2025), 25 pages. <https://doi.org/10.1145/3725324>
- [39] Gregory Kimball. 2025. Accelerating Large-Scale Data Analytics with GPU-Native Velox and NVIDIA cuDF. <https://velox-lib.io/blog/extending-velox-with-cudf/>.
- [40] Kinetica. (last accessed) 2025. The Fastest Database For AI and Analytics On Real-Time Data. [Online] Available from: <https://www.kinetica.com/>.
- [41] Dimitrios Koutsoukos, [others unspecified], and Co-authors. 2021. Tensors: An Abstraction for General Data Processing. *Proceedings of the VLDB Endowment* 14, 11 (2021), 1797–1809. <https://www.vldb.org/pvldb/vol14/p1797-koutsoukos.pdf> Investigates the use of tensor computation runtimes (TCRs) to support general data processing beyond ML.
- [42] Justin J. Levandoski, Per-Åke Larson, and Radu Stoica. 2013. Identifying hot and cold data in main-memory databases. In *29th IEEE International Conference on Data Engineering, ICDE 2013, Brisbane, Australia, April 8-12, 2013*, Christian S. Jensen, Christopher M. Jermaine, and Xiaofang Zhou (Eds.). IEEE Computer Society, Australia, 26–37. <https://doi.org/10.1109/ICDE.2013.6544811>
- [43] Jing Li, Hung-Wei Tseng, Chunbin Lin, Yannis Papakonstantinou, and Steven Swanson. 2016. HippogriffDB: balancing I/O and GPU bandwidth in big data

- analytics. *Proc. VLDB Endow.* 9, 14 (Oct. 2016), 1647–1658. <https://doi.org/10.14778/3007328.3007331>
- [44] Yanan Li, Bailu Ding, Ziyun Wei, Lukas M. Maas, Momin Al-Ghosien, Spyros Blanas, Nicolas Bruno, Carlo Curino, Matteo Interlandi, Craig Peeper, Kaushik Rajan, Surajit Chaudhuri, and Johannes Gehrke. 2025. Scaling GPU-Accelerated Databases beyond GPU Memory Size. *Proc. VLDB Endow.* 18, 11 (2025), 4518–4531. <https://www.vldb.org/pvldb/vol18/p4518-li.pdf>
- [45] Chunwei Liu, Anna Pavlenko, Matteo Interlandi, and Brandon Haynes. 2025. Data formats in analytical DBMSs: performance trade-offs and future directions. *VLDB J.* 34, 3 (2025), 30. <https://doi.org/10.1007/S00778-025-00911-1>
- [46] Shu Liu, Soujanya Ponnappalli, Shreya Shankar, Sepanta Zeighami, Alan Zhu, Shubham Agarwal, Ruiqi Chen, Samion Suwito, Shuo Yuan, Ion Stoica, Matei Zaharia, Alvin Cheung, Natacha Crooks, Joseph E. Gonzalez, and Aditya G. Parameswaran. 2025. Supporting Our AI Overlords: Redesigning Data Systems to be Agent-First. *CoRR abs/2509.00997* (2025). <https://doi.org/10.48550/ARXIV.2509.00997> arXiv:2509.00997
- [47] Alec Lu and Zhenman Fang. 2023. SQL2FPGA: Automatic Acceleration of SQL Query Processing on Modern CPU-FPGA Platforms. In *2023 IEEE 31st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, Marina Del Rey, California, 184–194. <https://doi.org/10.1109/FCCM57271.2023.00028>
- [48] Microsoft. 2025. Microsoft Fabric: A Unified Analytics Platform. <https://learn.microsoft.com/en-us/fabric/>. Accessed: 2025-10-10.
- [49] Microsoft. 2026. Godzilla: Large Scale Analytics on Multi-GPU Multi-nodes (Under preparation).
- [50] Microsoft. 2026. Scribe: a general purpose data-movement infrastructure for Microsoft Fabric (Under preparation).
- [51] Rene Mueller, Jens Teubner, and Gustavo Alonso. 2009. Data processing on FPGAs. *Proc. VLDB Endow.* 2, 1 (Aug. 2009), 910–921. <https://doi.org/10.14778/1687627.1687730>
- [52] Jacques Nadeau. 2022. Substrait: Rethinking DBMS Composability. In *1st International Workshop on Composable Data Management Systems, CDMS@VLDB 2022, Sydney, Australia, September 9, 2022*, Satyanarayana R. Valluri and Mohamed Zait (Eds.). VLDB, <https://dblp.org/rec/conf/vldb/Nadeau22.bib>.
- [53] Supun Nakandala, Karla Saur, Gyeong-In Yu, Konstantinos Karanasos, Carlo Curino, Markus Weimer, and Matteo Interlandi. 2020. A Tensor Compiler for Unified Machine Learning Prediction Serving. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, Virtual, 899–917. <https://www.usenix.org/conference/osdi20/presentation/nakandala>
- [54] Netezza. (last accessed) 2025. The Netezza Data Appliance Architecture: A Platform for High Performance Data Warehousing and Analytics. [Online] Available from: https://public.dhe.ibm.com/software/pdf/tr/roi/Netezza_Appliance_Architecture_WP.pdf.
- [55] Thomas Neumann. 2011. Efficiently Compiling Efficient Query Plans for Modern Hardware. *Proc. VLDB Endow.* 4, 9 (2011), 539–550. <https://doi.org/10.14778/2002938.2002940>
- [56] Jesse Noffsinger, Mark Patel, Pankaj Sachdeva, Arjita Bhan, Haley Chang, and Maria Goodpaster. 2025. The cost of compute: A \$7 trillion race to scale data centers. <https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/the-cost-of-compute-a-7-trillion-dollar-race-to-scale-data-centers> Estimates \$6.7T total data-center capex by 2030; \$5.2T attributable to AI workloads.
- [57] NVIDIA. 2025. Developing a Linux Kernel Module using GPUDirect RDMA. <https://docs.nvidia.com/cuda/gpudirect-rdma/index.html>.
- [58] NVIDIA. 2025. Infiniband. https://network.nvidia.com/pdf/whitepapers/IB_Intro_WP_190.pdf.
- [59] NVIDIA. 2025. NVIDIA NVLink. <https://www.nvidia.com/en-us/data-center/nvlink/>.
- [60] Muhsen Owaida, David Sidler, Kaan Kara, and Gustavo Alonso. 2017. Centaur: A Framework for Hybrid CPU-FPGA Databases. In *2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, Napa, California, 211–218. <https://doi.org/10.1109/FCCM.2017.37>
- [61] Pedro Pedreira, Orri Erling, Maria Basmanova, Kevin Wilfong, Laith Sakka, Krishna Pai, Wei He, and Biswapesh Chattopadhyay. 2022. Velox: Meta’s Unified Execution Engine. In *Proc. VLDB Endowment*. VLDB Endowment, Sydney, Australia, 3372–3384. <https://doi.org/10.14778/3554821.3554829>
- [62] Pedro Pedreira, Orri Erling, Konstantinos Karanasos, Scott Schneider, Wes McKinney, Satya R Valluri, Mohamed Zait, and Jacques Nadeau. 2023. The Composable Data Management System Manifesto. *Proceedings of the VLDB Endowment* 16, 10 (2023), 2679–2685. <https://doi.org/10.14778/3603581.3603604>
- [63] Blake Peltton, Adam Sapek, Ken Eguro, Daniel Lo, Alessandro Forin, Matt Humphrey, Jinwen Xi, David Cox, Rajas Karandikar, Johannes de Fine Licht, Evgeny Babin, Adrian Caulfield, and Doug Burger. 2024. Wavefront Threading Enables Effective High-Level Synthesis. *Proc. ACM Program. Lang.* 8, PLDI, Article 190 (June 2024), 25 pages. <https://doi.org/10.1145/3656420>
- [64] Nicole Hemsoth Prickett. 2016. Baidu Takes FPGA Approach to Accelerating SQL at Scale. [Online] Available from: <https://www.nextplatform.com/2016/08/24/baidu-takes-fpga-approach-accelerating-big-sql/>.
- [65] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data (Amsterdam, Netherlands) (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1981–1984. <https://doi.org/10.1145/3299869.3320212>
- [66] Kaushik Rajan, Sampath Rajendra, Momin Al-Ghosien, Nicolas Bruno, Carlo Curino, Matteo Interlandi, Yanan Li, Lukas M. Maas, Craig Peeper, Surajit Chaudhuri, and Johannes Gehrke. 2025–2026. Scalable GPU Acceleration of Scalar Functions in Analytical Databases: Compilation, Benchmarking, and Optimization (Under Submission).
- [67] RAPIDS. (last accessed) 2025. cuDF: A GPU DataFrame Library. [Online] Available from: <https://github.com/rapidsai/cudf>.
- [68] Viktor Rosenfeld, Sebastian Breß, and Volker Markl. 2022. Query Processing on Heterogeneous CPU/GPU Systems. *ACM Computing Surveys (CSUR)* 55, 1, Article 11 (Jan. 2022), 38 pages. <https://doi.org/10.1145/3485126>
- [69] Patricia G. Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data, Boston, Massachusetts, USA, May 30 - June 1, Philip A. Bernstein (Ed.)*. ACM, Boston, 23–34. <https://doi.org/10.1145/582095.582099>
- [70] Pavel Shamis, Manjunath Gorentla Venkata, M Graham Lopez, Matthew B Baker, Oscar Hernandez, Yossi Itigin, Mike Dubman, Gilad Shainer, Richard L Graham, Liran Liss, et al. 2015. UCX: an open source framework for HPC network APIs and beyond. In *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*. IEEE, IEEE, Santa Clara, California, 40–43.
- [71] Anil Shanbhag, Samuel Madden, and Xiangyao Yu. 2020. A Study of the Fundamental Performance Characteristics of GPUs and CPUs for Database Analytics. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (Portland, OR, USA) (SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 1617–1632. <https://doi.org/10.1145/3318464.3380595>
- [72] Anil Shanbhag, Bobbi W. Yogatama, Xiangyao Yu, and Samuel Madden. 2022. Tile-based Lightweight Integer Compression in GPU. In *Proceedings of the 2022 International Conference on Management of Data (Philadelphia, PA, USA) (SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 1390–1403. <https://doi.org/10.1145/3514221.3526132>
- [73] Speedata. 2025. Speedata: The Analytics Processing Unit. <https://www.speedata.io/>.
- [74] Substrait. 2025. Substrait Type Variations. https://substrait.io/types/type_variations/.
- [75] Nishith Vora et al. 2019. Apache Hudi: Ultra-Low Latency Data Management at Petabyte Scale. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands*. ACM, Amsterdam, 2025–2027.
- [76] Kaibo Wang, Kai Zhang, Yuan Yuan, Siyuan Ma, Rubao Lee, Xiaoning Ding, and Xiaodong Zhang. 2014. Concurrent analytical query processing with GPUs. *Proc. VLDB Endow.* 7, 11 (July 2014), 1011–1022. <https://doi.org/10.14778/2732967.2732976>
- [77] Louis Woods, Zsolt István, and Gustavo Alonso. 2013. Hybrid FPGA-accelerated SQL query processing. In *2013 23rd International Conference on Field Programmable Logic and Applications*. IEEE, Porto, Portugal. <https://doi.org/10.1109/FPL.2013.6645619>
- [78] Louis Woods, Zsolt István, and Gustavo Alonso. 2014. Ibox: an intelligent storage engine with support for advanced SQL offloading. *Proc. VLDB Endow.* 7, 11 (July 2014), 963–974. <https://doi.org/10.14778/2732967.2732972>
- [79] Bowen Wu, Wei Cui, Carlo Curino, Matteo Interlandi, and Rathijit Sen. 2025. Terabyte-Scale Analytics in the Blink of an Eye. *CoRR abs/2506.09226* (2025). <https://doi.org/10.48550/ARXIV.2506.09226> arXiv:2506.09226
- [80] Bobbi Yogatama, Yifei Yang, Kevin Kristensen, Devesh Sarda, Abigale Kim, Adrian Cockcroft, Yu Teng, Joshua Patterson, Gregory Kimball, Wes McKinney, Weiwei Gong, and Xiangyao Yu. 2025. Rethinking Analytical Processing in the GPU Era. arXiv:2508.04701 [cs.DB] <https://arxiv.org/abs/2508.04701>
- [81] Bobbi W. Yogatama, Weiwei Gong, and Xiangyao Yu. 2022. Orchestrating data placement and query execution in heterogeneous CPU-GPU DBMS. *Proc. VLDB Endow.* 15, 11 (July 2022), 2491–2503. <https://doi.org/10.14778/3551793.3551809>
- [82] Bobbi W. Yogatama, Weiwei Gong, and Xiangyao Yu. 2022. Orchestrating Data Placement and Query Execution in Heterogeneous CPU-GPU DBMS. *Proc. VLDB Endow.* 15, 11 (2022), 2491–2503. <https://doi.org/10.14778/3551793.3551809>
- [83] Yuan Yuan, Rubao Lee, and Xiaodong Zhang. 2013. The Yin and Yang of processing data warehousing queries on GPU devices. *Proc. VLDB Endow.* 6, 10 (Aug. 2013), 817–828. <https://doi.org/10.14778/2536206.2536210>
- [84] Daniel Ziener, Florian Bauer, Andreas Becher, Christopher Denny, Klaus Meyer-Wegener, Ute Schürfeld, Jürgen Teich, Jörg-Stephan Vogt, and Helmut Weber. 2016. FPGA-Based Dynamically Reconfigurable SQL Query Processing. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 9, 4, Article 25 (Aug. 2016), 24 pages. <https://doi.org/10.1145/2845087>